

Pandas & scikit-learn Patterns You'll See This Course — Session 3

Machine Learning in Geoscience — pre-session reference · read before Session 3

Sessions 1-2 were about getting honest data and an honest table. Session 3 is where models actually get trained — so the new patterns here are about the scikit-learn fit/predict lifecycle and about organizing results from many models × many horizons into something readable. If you've done the Session 2 self-check, the leakage-safe table itself should already feel familiar; this sheet covers what's new.

1. The model lifecycle: fit, then predict

`model.fit(X_train, y_train)`

Trains the model — it learns parameters from the training rows only. Nothing is returned that you need; the model object itself is updated in place.

`model.predict(X_val)`

Applies the already-trained model to new rows and returns predictions. Same shape as `estimator.transform()` from Session 2 — fit on train, apply elsewhere — just producing predictions instead of transformed features.

```
tree = DecisionTreeRegressor(max_depth=6)
tree.fit(X_train, y_train)           # learn
pred = tree.predict(X_val)           # apply, unseen data
```

`Pipeline([('scaler', StandardScaler()), ('model', LinearRegression())])`

Chains a preprocessing step and a model into a single object with its own `.fit()` / `.predict()`. Calling `pipe.fit(X_train, y_train)` fits the scaler on train, transforms train, then fits the model — all in the right order, so you can't accidentally leak by fitting the scaler on the wrong block.

2. Reading nested dictionaries

`DATA[H]['X_train']`

The course stores results in nested dicts keyed first by horizon, then by what you want: `DATA[6]['X_train']` is "the training features for the 6-hour horizon." Read it right-to-left: pick the horizon, then pick the piece.

```
BASE[H]['Persistence']['metrics']['MAE']
# ^horizon ^baseline name ^dict of scores ^one number
```

Looping to fill a dict

`for H in HORIZONS: RESULTS[H] = {...}` — the same block of code runs once per horizon and stores its own results under its own key. This is how one notebook handles 4 horizons without 4 copies of the code.

3. Turning results into a tidy table

`rows.append({...}); pd.DataFrame(rows)`

The standard way to build a results table: start an empty list, append one dict per row (e.g. one model × one horizon), then convert the whole list to a DataFrame at the end. Easier to get right than building the DataFrame column-by-column.

```
rows = []
for H in HORIZONS:
    for name, d in RESULTS[H].items():
        rows.append({'horizon_h': H, 'model': name, 'MAE':
d['metrics']['MAE']})
report = pd.DataFrame(rows)
```

`df.pivot(index=..., columns=..., values=...)`

Reshapes a long tidy table (one row per model+horizon) into a wide grid (models as rows, horizons as columns) — much easier to eyeball for "which model wins at which horizon."

`series.idxmin()` / `.idxmax()`

Returns the *label* of the smallest/largest value, not the value itself. Used to answer "which model had the lowest validation MAE at this horizon?" in one line.

```
best_name = pivot_mae.loc[MODEL_NAMES, H_PLOT].idxmin()
# -> e.g. 'Random Forest', not a number
```

4. A couple of smaller utilities

`series.mask(condition)`

Replaces values where the condition is True with `NaN` (opposite logic to `.where()`). Same effect as the `df.loc[cond, c] = np.nan` pattern from Session 1, written as one expression instead of an assignment.

```
raw[DATA_COLS] = raw[DATA_COLS].mask(raw[DATA_COLS] <= -900)
```

`zoom = slice(0, 24*10)`

A plain Python `slice` object, saved in a variable so it can be reused across several arrays/plots — here, "the first 10 days" of a validation block, applied consistently to observed, predicted and baseline series.

Before Session 3 — a 10-minute self-check

- On a toy dataset, fit any scikit-learn regressor with `.fit(X_train, y_train)` and get predictions with `.predict(X_val)`.
- Build a small list of dicts by hand, convert it with `pd.DataFrame(rows)`, and try `.pivot()` on it.
- If something above is still unclear, that's fine — flag it at the start of the session instead of getting stuck mid-lecture.

Machine Learning in Geoscience — Session 3 pre-read. This sheet covers recognition only; the course notebook is where these patterns get used for real, with full explanations in context.